arXiv:2402.13720v3 [cs.CL] 15 Oct 2024

Ouroboros: Generating Longer Drafts Phrase by Phrase for Faster Speculative Decoding

Weilin Zhao[1]*, Yuxiang Huang[1]*, Xu Han[1,2]†, Wang Xu[1], Chaojun Xiao[1], Xinrong Zhang[1], Yewei Fang[3], Kaihuo Zhang[3], Zhiyuan Liu[1]†, Maosong Sun[1]

[1]Department of Computer Science and Technology, Institute for Artificial Intelligence, Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing, China.
[2]Shanghai Artificial Intelligence Laboratory, Shanghai, China. [3]ModelBest Inc.
{zwl23,huang-yx21}@mails.tsinghua.edu.cn, {hanxu2022,liuzy}@tsinghua.edu.cn

Abstract

Speculative decoding is a widely used method that accelerates the generation process of large language models (LLMs) with no compromise in model performance. It achieves this goal by using an existing smaller model for drafting and then employing the target LLM to verify the draft in a low-cost parallel manner. Under such a drafting-verification framework, drafting efficiency has become a bottleneck in the final speedup of speculative decoding. Therefore, generating longer drafts at less cost can lead to better decoding speedup. To achieve this, we introduce Ouroboros, which can generate draft phrases to parallelize the drafting process and meanwhile lengthen drafts in a training-free manner. The experimental results on various typical text generation tasks show that Ouroboros can achieve speedups of up to $2.8\times$ over speculative decoding and $3.9\times$ over vanilla decoding, without fine-tuning draft and target models. The source code of Ouroboros is available at https://github.com/thunlp/Ouroboros.

1 Introduction

Benefiting from recent advances in parallel computing devices and distributed training algorithms (Shoeybi et al., 2019; Rasley et al., 2020), the training time of LLMs has been significantly shortened. However, it is still challenging to achieve parallelization in model inference because most LLMs rely on autoregressive generation mechanisms that exhibit sequential dependencies among generated tokens. For these autoregressive LLMs, they have to generate tokens one by one.

To accelerate the inference of LLMs, typical model compression methods such as quantization (Frantar et al., 2023; Lin et al., 2023; Kim et al., 2023) and pruning (Han et al., 2015; Wang et al., 2020; Xia et al., 2023b) may cause model performance degradation and sometimes even require non-negligible additional training costs. To losslessly accelerate the inference of LLMs, speculative decoding (Xia et al., 2023a; Leviathan et al., 2023; Chen et al., 2023a) has been proposed. Given a target LLM, speculative decoding selects a much smaller LLM as the draft model to generate multiple tokens as a draft. The target LLM then reviews the draft in parallel, accepting the longest accurate prefix while discarding the remaining draft tokens.

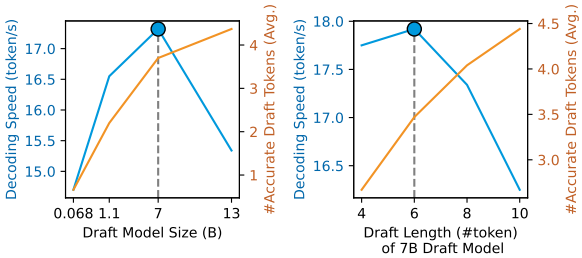

Figure 1: The trade-off between drafting efficiency and effectiveness. The figure illustrates the optimal speculative decoding speed of Llama-2-chat-70B on MT-Bench (Zheng et al., 2024) and the corresponding average number of accurate draft tokens at each iteration.

Under such a drafting-verification framework, how to efficiently generate long and accurate drafts has become a critical factor in accelerating LLM inference. As in Figure 1, we examine the trade-off between draft model size and speculative decoding speed. We observe that larger draft models tend to achieve higher draft accuracy, but the cost of generating drafts also rises, so that medium-sized models offer the best decoding speed. Besides, we investigate the trade-off between draft length and speculative decoding speed. Our findings indicate that the target model can accept more tokens per iteration on average when the draft is longer, but longer drafts introduce more forward passes required for the draft model, and thus, the opti-

* indicates equal contribution.
† indicates corresponding authors.

mal draft length in speculative decoding is not the largest one.

Based on the further analysis of the above pilot experimental results, it is not difficult to observe the limitations of speculative decoding regarding drafting efficiency: **(1) Insufficient Draft**. Drafting too short misses potential acceleration. However, because of the time overhead involved in drafting itself, generating long drafts may result in high costs when these drafts fail. If longer drafts can be generated more efficiently, the acceleration effect will be enhanced significantly. **(2) Underutilized Draft**. For current speculative decoding, tokens that are not accepted by the target model are completely discarded, resulting in a high failure cost when generating long drafts. In fact, some of the discarded tokens may contain useful information that could be utilized in future drafting iterations.

To overcome the above limitations, this paper introduces a more efficient decoding framework named Ouroboros, whose improvements over speculative decoding are as follows:

(1) Accelerating drafting via phrases. Given that model generation is memory-bound rather than computation-bound (Leviathan et al., 2023), drafting at the phrase level rather than the token level can make the drafting phase more efficient at producing longer drafts. Inspired by previous efforts such as lookahead decoding (Fu et al., 2023) that successfully employed phrases to accelerate the target model, Ouroboros adapts this method to enhance drafting efficiency. In subsequent sections, we will demonstrate that accelerating the draft model with phrases first and then using the draft to accelerate the target model will be more efficient than directly accelerating the target model with phrases.

(2) Lengthening drafts via phrases. Concatenating phrases can extend the draft even longer at a low cost. According to the last token of the draft, phrases starting with the last token can be used to extend the draft. By concatenating the draft and k different phrases, Ouroboros generates k longer drafts for the target model to verify, introducing almost zero additional costs.

(3) Generating phrases from verification. During the verification phase, Ouroboros filters out high-quality phrases from those discarded tokens in speculative decoding. These discarded phrases can be used to accelerate drafting in subsequent iterations.

(4) Reusing phrases from history contexts. We find that phrases generated from similar tasks can be reused to speed up drafting with each other. Ouroboros thus reuses phrases in history contexts to accelerate drafting in subsequent iterations.

Notably, Ouroboros does not require any additional training, and can be applied in all applications with speculative decoding. We implement Ouroboros and conduct sufficient experiments on various text generation tasks such as code generation (Chen et al., 2021; Austin et al., 2021), text summarization (See et al., 2017; Hermann et al., 2015) and machine translation (Bojar et al., 2016). The experimental results demonstrate that Ouroboros is completely lossless on task performance and can achieve significant inference acceleration without additional model fine-tuning. Compared with the recent competitive decoding methods, Ouroboros achieves speedups of up to 1.9× over lookahead decoding, up to 2.8× over speculative decoding, and up to 3.9× over naive autoregressive decoding.

2 Preliminary

For a target model $\mathcal{T}$ to accelerate inference, speculative decoding first finds a suitable draft model $\mathcal{S}$. Given an input prefix $x_{1\cdots n}$, speculative decoding uses the draft model $\mathcal{S}$ to generate a draft $d_{1\cdots\gamma}$ consisting of γ tokens in an autoregressive manner,

$$\begin{aligned} d_1 &= \arg\max_{d_1} P_{\mathcal{S}}(d_1 \mid x_{1\cdots n}), \\ d_i &= \arg\max_{d_i} P_{\mathcal{S}}(d_i \mid x_{1\cdots n}, d_{1\cdots i-1}),\ i = 2\cdots\gamma. \end{aligned} \tag{1}$$

Then, the target model $\mathcal{T}$ verifies the draft $d_{1\cdots\gamma}$, by inputting $d_{1\cdots\gamma}$ and conducting the next token prediction task using a single forward. Each verification result v_i is sampled from the next token prediction distribution of the target model,

$$\begin{aligned} v_1 &\sim P_{\mathcal{T}}(\cdot \mid x_{1\cdots n}), \\ v_i &\sim P_{\mathcal{T}}(\cdot \mid x_{1\cdots n}, d_{1\cdots i-1}),\ i = 2\cdots\gamma. \end{aligned} \tag{2}$$

There must exist an $A \in [0, \gamma]$ satisfying that

$$\begin{cases} v_{1\cdots A} = d_{1\cdots A}, \\ v_{A+1} \neq d_{A+1} \text{ or } A = \gamma. \end{cases} \tag{3}$$

Here, $v_{1\cdots A+1}$ will be the same as the result when the target model autoregressively generates $A+1$ tokens from x, because at this time,

$$\begin{aligned} v_1 &\sim P_{\mathcal{T}}(\cdot \mid x_{1\cdots n}), \\ v_i &\sim P_{\mathcal{T}}(\cdot \mid x_{1\cdots n}, v_{1\cdots i-1}),\ i = 2\cdots A+1. \end{aligned} \tag{4}$$

We call A the number of accurate tokens in the draft, $A+1$ tokens $d_{1\cdots A}, v_{A+1}$ are accepted as

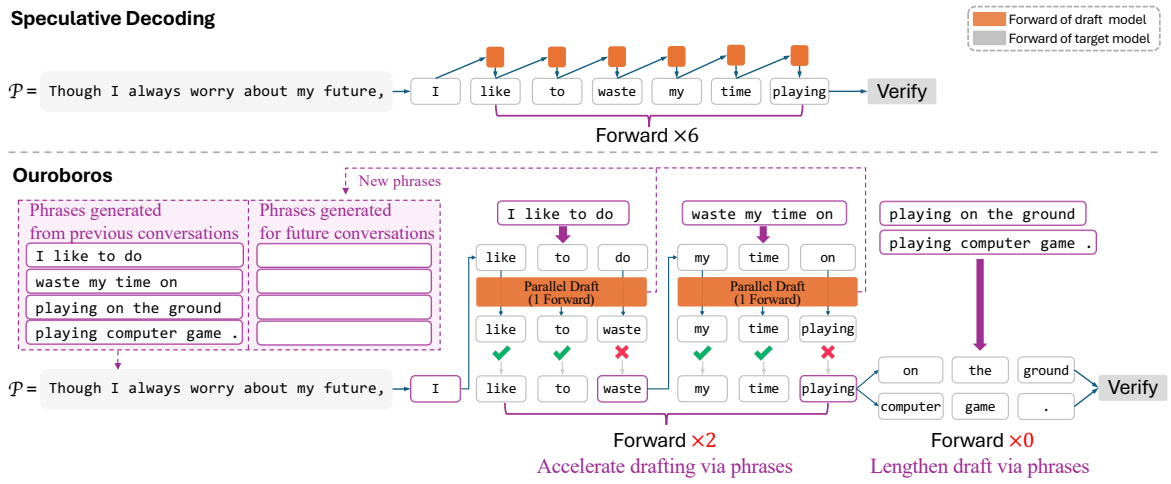

Figure 2: The framework of Ouroboros, which achieves better drafting efficiency than vanilla speculative decoding while also lengthening the drafts.

the generation result of the target model while the remaining draft tokens $d_{A+1\cdots\gamma}$ are discarded. Please note that the derivation above differs slightly from the original speculative decoding (Chen et al., 2023a): we set the temperature for the random sampling in the draft model to 0 for a clearer explanation, but this would not affect the correctness of the target model.

We define $A(\gamma)$ as the average number of accurate tokens when the draft model is $\mathcal{S}$ and the draft length is γ. The speedup of speculative decoding is calculated as follows (Leviathan et al., 2023), where $t_{\mathcal{S}}$ and $t_{\mathcal{T}}$ are the forward time of the draft model and the target model, respectively,

$$\frac{[A(\gamma)+1]\cdot t_{\mathcal{T}}}{\gamma\cdot t_{\mathcal{S}}+t_{\mathcal{T}}}, \tag{5}$$

where the numerator is the time for the vanilla generation of the target model, and the denominator is the time of the speculative decoding.

3 Method

In Ouroboros, we first accelerate drafting via phrases, so that we can generate γ tokens with fewer forward passes of the draft model, and we denote this reduction ratio as c. Subsequently, due to the fact that function $A(\gamma)$ is monotonically non-decreasing, we lengthen drafts via phrases, enabling the generation of more tokens without the need for additional forward passes. We denote this costless extended length as β. Following these two optimization directions, Eq. (5) becomes

$$\frac{[A(\gamma+\beta)+1]\cdot t_{\mathcal{T}}}{\frac{1}{c}\gamma\cdot t_{\mathcal{S}}+t_{\mathcal{T}}}. \tag{6}$$

We further enlarge c by heuristically generating phrases from verification results and reusing phrases from historically generated contexts, which can lead to better drafting efficiency. Next, we will delve into the details for each part of Ouroboros.

3.1 Accelerating Drafting via Phrases

We get inspiration from lookahead decoding (Fu et al., 2023), which uses phrases to directly accelerate the target model $\mathcal{T}$. However, each round of phrase drafting requires a forward pass of the target model $\mathcal{T}$ to verify the draft, limiting the whole acceleration effect of lookahead decoding. Different from lookahead decoding, we use phrases to indirectly accelerate the target model $\mathcal{T}$ through a draft model $\mathcal{S}$, allowing each forward pass of the target model to simultaneously verify multiple rounds of phrases, achieving a better acceleration. As shown in Figure 2, in Ouroboros, the drafting process of the draft model is performed phrase by phrase instead of token by token. Multiple new phrases are generated in parallel during each forward of the draft model. We will not go into detail how to generate new draft phrases in parallel due to space constraints, and can refer to Fu et al. (2023).

3.2 Lengthening Drafts via Phrases

Given that model generation is memory-bound, the time it takes for the target model to verify dozens of tokens using a single forward is not much different from the time spent on verifying a single token (Leviathan et al., 2023). Therefore, we propose to use phrases to heuristically extend drafts since phrase concatenation is nearly zero-cost.

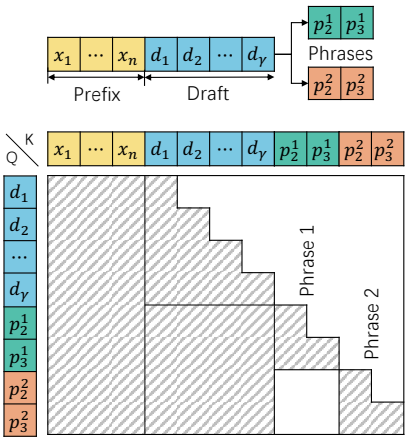

Figure 3: The customized attention masking mechanism for verifying lengthened drafts.

Trying multiple phrases to get multiple longer drafts can increase the probability that one of these phrases passes the verification of the target model. But the cost of verifying these longer drafts one by one is also unbearable. Inspired by Cai et al. (2024), we construct a sophisticated Transformer attention masking mechanism to complete the verification of all longer drafts with only one forward pass of the target model $\mathcal{T}$, as shown in Figure 3.

Specifically, given a draft $d_{1\cdots\gamma}$, we select out K phrases $p^1_{1\cdots\beta}, \cdots, p^K_{1\cdots\beta}$ starting with the token d_γ, i.e., $p^1_1 = p^2_1 = \cdots = p^K_1 = d_\gamma$. The K lengthened drafts are

$$\begin{cases} d_{1\cdots\gamma}, p^1_{2\cdots\beta}, \\ d_{1\cdots\gamma}, p^2_{2\cdots\beta}, \\ \quad\vdots \\ d_{1\cdots\gamma}, p^K_{2\cdots\beta}. \end{cases} \tag{7}$$

The target model then calculates v^j_i to verify p^j_i for all $j = 2\cdots K, i = 2\cdots\beta+1$ in Eq. (8). All v^j_i are computed in parallel by a single forward using the attention masking mechanism in Figure 3.

$$v^j_i \sim P_{\mathcal{T}}(\cdot \mid x_{1\cdots n}, d_{1\cdots\gamma}, p^j_{2\cdots i-1}) \tag{8}$$

Similar to Eq. (3), we define $\hat{A}^j$ satisfying

$$\begin{cases} v^j_{2\cdots\hat{A}^j} = p^j_{2\cdots\hat{A}^j}, \\ v^j_{\hat{A}^j+1} \neq p^j_{\hat{A}^j+1} \text{ or } \hat{A}^j = \beta. \end{cases} \tag{9}$$

Once the draft $d_{1\cdots\gamma}$ is fully accepted by the target model, we use the phrase p^j with the largest $\hat{A}^j$ to enlarge the number of accepted tokens. More specifically, $d_{1\cdots\gamma}, p^j_{2\cdots\hat{A}^j}, v^j_{\hat{A}^j+1}$ will be accepted.

Yi-base 34B/6B	MBPP	CNN/DM	WMT-16
A	12.7	8.4	5.3
#Match	18.9	17.8	17.0

Table 1: Average matched tokens compared with the number of accepted tokens. Experiment on Yi-base 34B/6B (Young et al., 2024) on three datasets, MBPP (Austin et al., 2021), CNN/DM (See et al., 2017; Hermann et al., 2015) and WMT16 (Bojar et al., 2016) using Speculative Decoding, with draft length γ = 20.

3.3 Generating Phrases from Verification

In speculative decoding, the cost when a draft fails is relatively high. If draft $d_{1\cdots\gamma}$ is not fully accepted, traditional verification only accepts the correct draft prefix $d_{1\cdots A}$ and discards $d_{A+1\cdots\gamma}$, as in Section 2. To evaluate the information in these discarded draft tokens, we define #Match$(d_{1\cdots\gamma}, v_{1\cdots\gamma})$ the number of the matched tokens of the draft and the verification result,

$$\text{\#Match}(d_{1\cdots\gamma}, v_{1\cdots\gamma}) = \sum_{i=1}^{\gamma} [d_i = v_i], \tag{10}$$

where $[d_i = v_i] = 1$ when $d_i = v_i$ and 0 otherwise.

As in Table 1, we find that #Match is much larger than the length of the accurate prefix A. We view some specific cases and find that sometimes the emergence of this situation is caused by the misplacement of the generation, as shown in Figure 4. Therefore, we propose to filter out those sub-segments of the discarded draft that match the verification results of the target model, as shown in Figure 4. We then insert those segments into the phrases pool to accelerate future drafts.

As in Section 3.2, if draft $d_{1\cdots\gamma}$ is fully accepted, only one phrase suffix p^j is used, leaving all other tried phrases unused. We suppose the verification result $v^o_{2\cdots\beta}$ fixes some errors in the phrase $p^o_{2\cdots\beta}$ for all $o \neq j$: we use $p^o_1, v^o_{2\cdots\beta}$ to replace the phrase $p^o_{1\cdots\beta}$ in the phrase pool.

3.4 Reusing Phrases from History Contexts

In real-world scenarios, the adjacent conversations from users may exhibit similarities. Leveraging these similarities, we can further enhance the speed of the model inference. Different from lookahead decoding that cleanup phrases from history contexts when conducting the generation of subsequent input queries, we propose to reuse phrases generated from previous generations to further accelerate the drafting process.

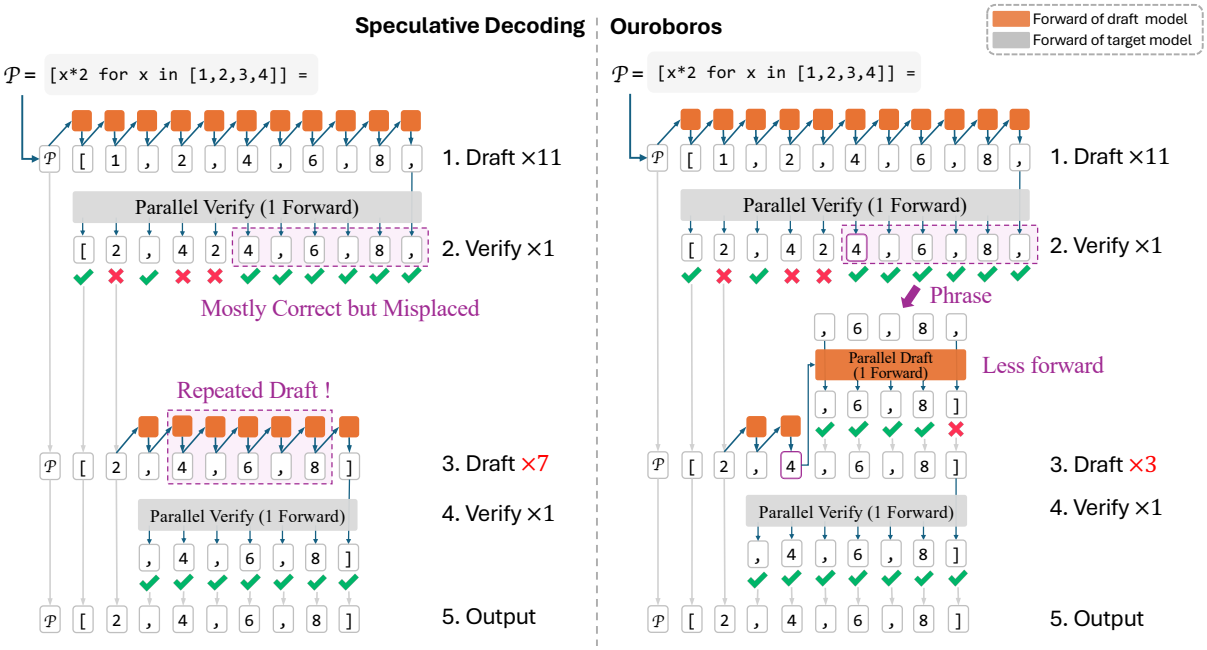

Figure 4: The illustration of generating phrases from the verification results of the target model.

3.5 The Advantages of the Training-free Framework

Ouroboros is entirely training-free. We have not employed methods like model distillation to increase the function $A(\gamma)$ or model compression to decrease t_S in Eq. (5). In terms of phrase generation, all our phrases generation strategies are heuristic. All phrases are gradually accumulated during the generation process of models, without prior preparation on a large-scale corpus. All these mean that, given a draft model in any speculative decoding method, we can use Ouroboros to help these methods achieve further speedup without introducing additional costs.

4 Experiments

This section focuses on evaluating Ouroboros on various text generation tasks to demonstrate the efficiency and effectiveness of Ouroboros.

4.1 Experimental Settings

In order to evaluate the overall acceleration caused by Ouroboros, we evaluate Ouroboros on various typical text generation tasks, including code generation, arithmetic reasoning, document summarization, and machine translation.

Datasets. For code generation, we evaluate Ouroboros on HumanEval (Chen et al., 2021) and the validation set of MBPP (Austin et al., 2021). There are 164 entries in HumanEval and they are composed of a text prompt and a prefix of Python function. The validation set of MBPP has 90 entries, in which the whole function is expected to be predicted with a given text prompt and test cases. The maximum generation lengths on HumanEval and MBPP are set to 512. For arithmetic reasoning, document summarization and machine translation, we evaluate our method on GSM8K (Cobbe et al., 2021), CNN/DM (See et al., 2017; Hermann et al., 2015) and WMT16 (Bojar et al., 2016), respectively. We randomly sample 100 entries from GSM8K and CNN/DM, and sample 100 entries from the German-to-English translation subset of WMT16. The maximum generation lengths on GSM8k, CNN/DM and WMT16 are respectively set to 256, 128, and 64.

Models. For HumanEval and MBPP, we use Yi-base-34B/6B (Young et al., 2024), DeepSeek-coder-instruct-33b/6.7B (Bi et al., 2024) and CodeLlama-instruct-34B/7B (Roziere et al., 2023) as the target/draft models for our experiments. For GSM8K, CNN/DM and WMT16, we use Yi-base-34B/6B and Llama-2-chat-70B/7B (Touvron et al., 2023). We use the larger model as the target model and the smaller one for drafting. All these models are recently representative and popular LLMs.

Evaluation Methods. The baselines of our experiments are vanilla autoregressive decoding, speculative decoding (Leviathan et al., 2023; Chen et al., 2023a), and lookahead decoding (Fu et al., 2023). We report the decoding speed (token/s) and

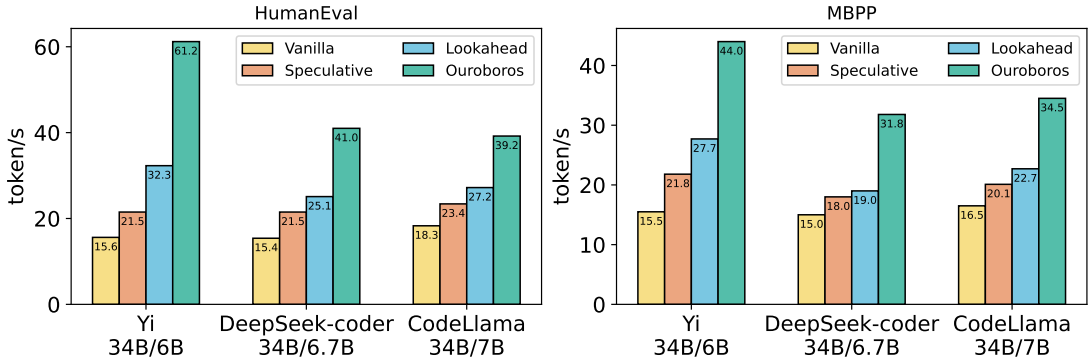

Figure 5: The greedy decoding speed (token/s) on HumanEval and MBPP.

Task	Algorithm	Yi 34B/6B		Llama-2 70B/7B	
		token/s	speedup	token/s	speedup
GSM8k	Vanilla	15.33	1.00×	8.96	1.00×
	Speculative	16.99	1.11×	16.86	1.88×
	Lookahead	25.14	1.64×	13.77	1.54×
	Ouroboros	**28.23**	**1.84**×	**24.03**	**2.68**×
CNN/DM	Vanilla	14.62	1.00	8.12	1.00
	Speculative	17.82	1.22×	12.77	1.57×
	Lookahead	18.77	1.28×	9.47	1.17×
	Ouroboros	**22.65**	**1.55**×	**14.67**	**1.81**×
WMT16	Vanilla	14.78	1.00×	9.52	1.00×
	Speculative	17.48	1.18×	14.72	1.55×
	Lookahead	17.98	1.22×	14.65	1.54×
	Ouroboros	**19.94**	**1.35**×	**19.27**	**2.02**×

Table 2: The greedy decoding speed (token/s) and speedup ratio on GSM8K, CNN/DM and WMT16.

Task	Algorithm	Greedy	Random	
		(temp=0.0)	temp=0.5	temp=1.0
GSM8k	Vanilla	8.96	8.97	8.96
	Speculative	16.86	14.99	14.11
	Lookahead	13.77	13.56	13.46
	Ouroboros	**24.03**	**22.04**	**19.27**
CNN/DM	Vanilla	8.12	8.83	8.30
	Speculative	12.77	13.43	13.75
	Lookahead	9.47	9.31	9.47
	Ouroboros	**14.67**	**14.97**	**14.23**
WMT16	Vanilla	9.52	9.75	9.75
	Speculative	14.72	13.91	14.28
	Lookahead	14.65	12.05	11.91
	Ouroboros	**19.27**	**19.95**	**19.33**

Table 3: The random sampling speed (token/s) compared with the greedy decoding speed (token/s), tested on Llama-2-chat 70B/7B. "temp" means the temperature used in the random sampling scenario.

the speedup ratio compared with vanilla autoregressive decoding. More hyperparameters of our experiments are included in Appendix B.

Hardware and Implementation. All experiments are performed on 2 × NVIDIA 80GB A800 GPU with NVLINK × 8 interconnected. The CPU used is the Intel(R) Xeon(R) Platinum 8350C. We use the Huggingface transformers package to conduct automatic model parallelism for our methods and all baselines.

4.2 Overall Results

We conduct experiments in both greedy decoding and random sampling scenarios. The experimental results are listed as follows.

Greedy decoding. As shown in Figure 5, in the greedy generation scenario, Ouroboros outperforms vanilla greedy decoding, lookahead decoding, and speculative decoding under all backbone models and dataset configurations. Ouroboros can achieve up to 61.2 token/s generation speed with the max generation length of 512, which achieves speedups of 3.9× compared to greedy decoding, 2.8× compared to speculative decoding, and 1.9× compared to lookahead decoding. Table 2 shows that Ouroboros can also get substantial speedups on typical natural language tasks, where lookahead decoding and speculative decoding can only achieve limited acceleration.

Random sampling. We test Ouroboros in the random sampling scenario using Llama-2-chat-70B/7B, considering that those models after SFT are more suitable for well understanding rich semantics distributed in human natural languages and generating diverse outputs. For the sampling hyperparameters, the generation temperature is set among 0.5 and 1.0, and top-p is set to 0.8. Table 3 shows that Ouroboros can also be applied to random sampling, and the speedup over baseline methods are not much different from the observations in the greedy decoding scenario.

Method	token/s
Baseline	21.46
+ Accelerating drafting via phrases	49.90
+ Lengthening drafts via phrases	55.92
+ Generate phrases from verification	58.18
+ Reuse phrases from history context	61.20

Table 4: The ablation studies of each component in Ouroboros on HumanEval using Yi 34B/6B.

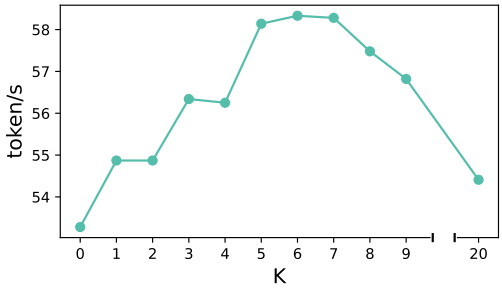

Figure 6: The effect of selecting K phrases for Eq. (7), tested on HumanEval using Yi 34B/6B without reusing phrases from history contexts.

4.3 Ablation Studies and Analyses

In this section, to give a deeper insight into how Ouroboros achieves higher generation speed, we conduct ablation studies and analyses to answer the following questions.

What are the effects of each component? To demonstrate the specific speedup introduced by each mechanism, the ablation results are in Table 4. In the table, each component could bring a speedup gain, while accelerating drafting and lengthening the draft are the most effective ones.

How many phrases are needed to lengthen the draft? Verifying too many phrases might slow down the verification in real-model inference scenarios. Here comes a trade-off between more phrases for possible speedup and slower verification. In Figure 6, there exists a best K value, fewer or more phrases cause slower decoding speed.

Other ablation experiments. The results of Ouroboros on different hyperparameter and length settings are show in Appendix B and Appendix E, respectively. The ablation of phrases reusing on different context locality are in Appendix F.

4.4 Comparing with Training-based Methods

We compare our training-free Ouroboros with the state-of-the-art training-based method Eagle (Li et al., 2024) on Spec-Bench (Xia et al., 2024). The main idea of Eagle is to specifically train a tiny draft model under their newly-designed model architecture and generate tree-style drafts for the target LLM. In order to accelerate Llama-2-chat-70B, it trains a 1B draft model by distilling from Llama-2-chat-70B. The results in Table 5 show that, although Eagle trains a much smaller draft model with $\frac{1}{7}$ of our scale to draft, it can only achieve the speed slightly faster than our training-free method. This is because the much smaller model significantly reduces the number of draft tokens accepted by the target model in each iteration, i.e., lossing draft effectiveness. We look back at the training-based methods including Eagle, they pursue ultimate drafting speed at the expense of lossing draft accuracy. We, on the other hand, optimize the drafting speed while keeping the accuracy unchanged. Currently, Eagle cannot be integrated into Ouroboros since the special model architecture of their draft model only supports autoregressive token-level drafting. We believe that our method can find a better balance between drafting speed and accuracy when combined with training, which will be our future work.

Spec-Bench	Ouroboros		Eagle	
	token/s	#accept	token/s	#accept
MT-Bench	24.23	**5.16**	**28.20**	3.52
Translation	18.91	**3.92**	**24.23**	3.16
Summarization	19.91	**4.93**	**22.03**	3.16
QA	21.63	**4.67**	**24.83**	3.23
Math Reasoning	25.39	**4.95**	**29.90**	3.81
RAG	19.00	**5.43**	**20.56**	3.54
Average	21.51	**4.96**	**24.96**	3.48

Table 5: The speed (token/s) comparing with training-based methods, tested on Llama-2-chat-70B. "#accept" means the number of draft tokens accepted by the target model in each iteration (on average). Ouroboros uses Llama-2-chat-7B as the draft model while Eagle uses its trained 1B model to draft.

4.5 Comparing with Phrases-based Methods

We compare Ouroboros with other phrases-based method PLD (Saxena, 2023) and REST (He et al., 2024) on Spec-Bench (Xia et al., 2024). These methods retrieve phrases from prompts or documents as drafts for the target model to verify. Ouroboros outperforms these baselines by a large margin in Table 6, which shows the effectiveness of using a draft model as an intermediary to filter away low-quality phrases before providing them to the target model.

Spec-Bench	Ouroboros		PLD		REST	
	token/s	#accept	token/s	#accept	token/s	#accept
MT-Bench	**24.23**	**5.16**	13.95	1.45	14.93	1.94
Translation	**18.91**	**3.92**	12.99	1.41	13.02	1.57
Summarization	**19.91**	**4.93**	16.98	1.97	12.60	1.69
QA	**21.63**	**4.67**	11.33	1.26	16.10	1.96
Math Reasoning	**25.39**	**4.95**	15.86	1.75	13.07	1.60
RAG	**19.00**	**5.43**	15.51	1.64	13.28	1.91
Average	**21.51**	**4.96**	14.44	1.51	13.83	1.83

Table 6: The speed (token/s) comparing with phrases-based methods, tested on Llama-2-chat-70B. "#accept" means the number of draft tokens accepted by the target model in each iteration (on average).

5 Related Work

This section introduces existing efforts for efficient LLM inference, including efficient decoding, efficient implementation and model compression.

5.1 Efficient Decoding

To alleviate efficiency issues caused by autoregressive decoding, non-autoregressive decoding methods have been proposed. Instead of generating tokens one by one, non-autoregressive methods generate multiple tokens in parallel at a time (Wei et al., 2019; Guo et al., 2020; Ghazvininejad et al., 2019). These non-autoregressive methods bring an improvement in inference efficiency and also significantly hurt model performance. To this end, drafting-then-verifying methods have been proposed (Stern et al., 2018; Xia et al., 2023a; Leviathan et al., 2023; Chen et al., 2023a). The drafting-then-verifying methods avoid LLMs from serial generation and instead use them to verify drafts in a non-autoregressive parallel manner, which do not reduce model performance and significantly accelerate inference.

The key to drafting-then-verifying methods is to generate drafts quickly and well.

(1) Methods such as speculative decoding (Xia et al., 2023a; Leviathan et al., 2023; Chen et al., 2023a) use a model smaller than the target model to generate drafts. To further align the draft model with the target model, distillation techniques are applied (Miao et al., 2023; Zhou et al., 2023). Even much smaller models can be used as a draft to speed up the draft model, forming multi-staged speculative decoding (Spector and Re, 2023; Chen et al., 2023b). Apart from using different models, Self-Speculative (Zhang et al., 2024a) and PPD (Yang et al., 2023b) select part of the model layers as the draft model, while these methods requires extra training or pre-processing.

(2) Some other efforts explore to use the target model itself to efficiently generate drafts. Blockwise (Stern et al., 2018) and Medusa (Cai et al., 2024) train multiple extra output heads on top of LLMs and fine-tune heads to generate multiple draft tokens in parallel. Parallel Decoding (Santilli et al., 2023) and PaSS (Monea et al., 2023) add auxiliary input suffixes such as padding tokens or learnable padding tokens to generate draft output suffixes. Lookahead Decoding (Fu et al., 2023) generates n-gram pools using Jacobi-iteration and uses those n-grams starting with the last generated token as drafts.

(3) Other methods explore to retrieve phrases from previous prompts or existing documents, such as PLD (Saxena, 2023), LLMA (Yang et al., 2023a), and REST (He et al., 2024). These methods use the target model to directly verify phrases, incurring high failure costs on each draft trial. However, we are the first to use a draft model as an intermediary to filter away low-quality phrases before providing them to the target model.

Both the above methods are insufficiently efficient, so we first introduce a small model for drafting and then further accelerate the draft process of the small model by using phrases. We list various possibilities for the sources of phrases (from draft model, from verification, and from historical contexts) in our method and conduct ablation experiments for each source. This indicates the possibility that other phrase sources (Saxena, 2023; Yang et al., 2023a; He et al., 2024) can be integrated into our framework to achieve further speedup.

Besides better generating drafts, traditional triangular attention masking can only verify one draft sentence using a complete forward pass. Tree-style verification (Miao et al., 2023; Cai et al., 2024; Fu et al., 2023) designs specific attention masking to verify multiple possible drafts at a time.

5.2 Efficient Implementation

The most direct solution to achieving efficient LLM inference is implementing LLMs efficiently to take advantage of hardware devices (such as GPUs). FlashDecoding (Dao et al., 2023) accelerates Transformer attention computation within LLMs by partitioning the decoding sequence into multiple small blocks and performing block-wise computation in fast GPU SRAM instead of GPU HBM. PageAttention (Kwon et al., 2023) using paged virtual memory management to organize the decoding cache during the decoding process, thereby effectively

utilizing GPU memory bandwidth to reduce the memory access overhead of inference. Tensor Parallelism (Shoeybi et al., 2019) accelerates inference by sharding matrices into distributed GPUs and performing matrix multiplications in a distributed manner. Some efforts implement LLMs by optimizing underlying operators (Nvidia, a; Wang et al., 2021; Nvidia, b) and achieve promising results. Note that these efficient implementations of LLMs and Ouroboros are orthogonal. Combining efficient implementation methods and Ouroboros can achieve more significant inference acceleration.

5.3 Model Compression

Model compression methods are proposed to reduce the number of operations needed for model execution. Structure pruning (Fan et al., 2020; Wang et al., 2020; Zhang et al., 2021; Xia et al., 2022) and unstructured pruning (Han et al., 2015; Chen et al., 2020; Xu et al., 2021) elimate non-essential parameters. Quantization (Zafrir et al., 2019; Frantar et al., 2023; Lin et al., 2023; Kim et al., 2023; Stock et al., 2021) methods quantize parameters into low-bit representations. Distillation (Hinton et al., 2015; Sun et al., 2019; Jiao et al., 2020; Liu et al., 2022; Park et al., 2021) methods are used to help align those compressed models to their original versions to maintain model performance. Early-exiting (Elbayad et al., 2019; Bae et al., 2023) methods end the inference process when the output result in shallow layers reaches the confidence threshold. MoEfication (Zhang et al., 2022; Song et al., 2023) turns a dense model into a sparse activated model. Model compression and Ouroboros are also orthogonal and can be integrated for further acceleration.

6 Conclusion

In this paper, we propose a practical algorithm Ouroboros to improve drafting efficiency for speculative decoding. We generate phrases in a heuristic manner and use phrases to help draft models accelerate drafting and lengthen drafts. Our experiments verify that, compared to typical baselines vanilla speculative decoding and lookahead decoding, Ouroboros respectively achieves speedups of 2.8× and 1.9× and does not affect the generation quality at all. Our method is completely training-free, to make it easier to adopt for the users of speculative decoding. And the training-based version of Ouroboros towards extreme acceleration will be our next work.

Limitations

We only focus on decoder-only model structure in the experiment. However, Ouroboros is an extendable framework, which can be applied to various model structures. Our method currently focuses on optimization in training-free scenarios. If the consistency of large and small models is improved through training in the future, our method can achieve higher acceleration. Both efficient implementation and model compression are orthogonal to our method, and can be combined for further speedup.

We only focus on the single query scenario. The application of speculative sampling in the batched inference scenario is not within the scope of this paper and can refer to (Liu et al., 2024; Qian et al., 2024; Chen et al., 2024).

Ethical Considerations

Our method has no potential risk since we are training-free and does not affect the generation results at all.

Acknowledgments

This work was supported by the National Key R&D Program of China (2022ZD0160501), Institute Guo Qiang at Tsinghua University. Yuxiang Huang is supported by Tsinghua University Initiative Scientific Research Program (Student Academic Research Advancement Program). We acknowledge valuable discussions with Yuhui Li, the author of EAGLE.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Sangmin Bae, Jongwoo Ko, Hwanjun Song, and Se-Young Yun. 2023. Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding. In *Proceedings of EMNLP*.

Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*.

Ond rej Bojar, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, Varvara Logacheva, Christof Monz, Matteo Negri, Aurelie Neveol, Mariana Neves, Martin Popel, Matt Post, Raphael Rubino, Carolina Scarton, Lucia Specia, Marco Turchi, Karin Verspoor, and Marcos Zampieri. 2016. Findings of the 2016 conference on machine translation. In *Proceedings of the First Conference on Machine Translation*, pages 131–198.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv: 2401.10774*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023a. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Jian Chen, Vashisth Tiwari, Ranajoy Sadhukhan, Zhuoming Chen, Jinyuan Shi, Ian En-Hsu Yen, and Beidi Chen. 2024. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. *arXiv preprint arXiv:2408.11049*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Tianlong Chen, Jonathan Frankle, Shiyu Chang, Sijia Liu, Yang Zhang, Zhangyang Wang, and Michael Carbin. 2020. The lottery ticket hypothesis for pre-trained bert networks. In *Proceedings of NeurIPS*, pages 15834–15846.

Ziyi Chen, Xiaocong Yang, Jiacheng Lin, Chenkai Sun, Jie Huang, and Kevin Chen-Chuan Chang. 2023b. Cascade speculative drafting for even faster llm inference. *arXiv preprint arXiv:2312.11462*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-decoding for long-context inference.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2019. Depth-adaptive transformer. In *Proceedings of ICLR*.

Angela Fan, Edouard Grave, and Armand Joulin. 2020. Reducing transformer depth on demand with structured dropout. In *Proceedings of ICLR*.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. Gptq: Accurate quantization for generative pre-trained transformers. In *Proceedings of ICLR*.

Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2023. Breaking the sequential dependency of llm inference using lookahead decoding.

Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-predict: Parallel decoding of conditional masked language models. *arXiv preprint arXiv:1904.09324*.

Junliang Guo, Linli Xu, and Enhong Chen. 2020. Jointly masked sequence-to-sequence model for non-autoregressive neural machine translation. In *Proceedings of ACL*, pages 376–385.

Song Han, Jeff Pool, John Tran, and William J. Dally. 2015. Learning both weights and connections for efficient neural network. In *Proceedings of NeurIPS*, pages 1135–1143.

Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. 2024. REST: Retrieval-based speculative decoding. In *Proceedings of NAACL*, pages 1582–1595.

Karl Moritz Hermann, Tomás Kociský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching machines to read and comprehend. In *Proceedings of NeurIPS*, pages 1693–1701.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for natural language understanding. In *Findings of EMNLP*, pages 4163–4174.

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2023. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of SOSP*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of ICML*, pages 19274–19286.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of ICML*.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*.

Chang Liu, Chongyang Tao, Jiazhan Feng, and Dongyan Zhao. 2022. Multi-granularity structural knowledge distillation for language model compression. In *Proceedings of ACL*, pages 1001–1011.

Xiaoxuan Liu, Cade Daniel, Langxiang Hu, Woosuk Kwon, Zhuohan Li, Xiangxi Mo, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2024. Optimizing speculative decoding for serving large language models using goodput. *arXiv preprint arXiv:2406.14066*.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2023. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*.

Giovanni Monea, Armand Joulin, and Edouard Grave. 2023. Pass: Parallel speculative sampling. *arXiv preprint arXiv:2311.13581*.

Nvidia. a. Fastertransformer.

Nvidia. b. Tensorrt-llm.

Geondo Park, Gyeongman Kim, and Eunho Yang. 2021. Distilling linguistic context for language model compression. In *Proceedings of EMNLP*, pages 364–378.

Haifeng Qian, Sujan Kumar Gonugondla, Sungsoo Ha, Mingyue Shang, Sanjay Krishna Gouda, Ramesh Nallapati, Sudipta Sengupta, Xiaofei Ma, and Anoop Deoras. 2024. Bass: Batched attention-optimized speculative sampling. *arXiv preprint arXiv:2404.15778*.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of SIGKDD*, pages 3505–3506.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Andrea Santilli, Silvio Severino, Emilian Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, and Emanuele Rodolà. 2023. Accelerating transformer inference for translation via parallel decoding. *arXiv preprint arXiv:2305.10427*.

Apoorv Saxena. 2023. Prompt lookup decoding.

Abigail See, Peter J. Liu, and Christopher D. Manning. 2017. Get to the point: Summarization with pointer-generator networks. In *Proceedings of ACL*, pages 1073–1083.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.

Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. Powerinfer: Fast large language model serving with a consumer-grade gpu. *arXiv preprint arXiv:2312.12456*.

Benjamin Spector and Chris Re. 2023. Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623*.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. volume 31.

Pierre Stock, Angela Fan, Benjamin Graham, Edouard Grave, Rémi Gribonval, Herve Jegou, and Armand Joulin. 2021. Training with quantization noise for extreme model compression. In *Proceedings of ICLR*.

Siqi Sun, Yu Cheng, Zhe Gan, and Jingjing Liu. 2019. Patient knowledge distillation for BERT model compression. In *Proceedings EMNLP-IJCNLP*, pages 4323–4332.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Xiaohui Wang, Ying Xiong, Yang Wei, Mingxuan Wang, and Lei Li. 2021. LightSeq: A high performance inference library for transformers. In *Proceedings of NAACL*, pages 113–120.

Ziheng Wang, Jeremy Wohlwend, and Tao Lei. 2020. Structured pruning of large language models. In *Proceedings of EMNLP*, pages 6151–6162.

Bingzhen Wei, Mingxuan Wang, Hao Zhou, Junyang Lin, Jun Xie, and Xu Sun. 2019. Imitation learning for non-autoregressive neural machine translation. *arXiv preprint arXiv:1906.02041*.

Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. 2023a. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In *Proceedings of EMNLP*, pages 3909–3925.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2023b. Sheared llama: Accelerating language model pre-training via structured pruning. *arXiv preprint arXiv:2310.06694*.

Mengzhou Xia, Zexuan Zhong, and Danqi Chen. 2022. Structured pruning learns compact and accurate models. In *Proceedings of ACL*, pages 1513–1528.

Dongkuan Xu, Ian En-Hsu Yen, Jinxi Zhao, and Zhibin Xiao. 2021. Rethinking network pruning – under the pre-train and fine-tune paradigm. In *Proceedings of NAACL*, pages 2376–2382.

Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023a. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*.

Seongjun Yang, Gibbeum Lee, Jaewoong Cho, Dimitris Papailiopoulos, and Kangwook Lee. 2023b. Predictive pipelined decoding: A compute-latency trade-off for exact llm decoding. *arXiv preprint arXiv:2307.05908*.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. 2024. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652*.

Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8bert: Quantized 8bit bert. In *Proceedings of EMC2-NIPS*, pages 36–39.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024a. Draft& verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of ACL*, pages 11263–11282.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024b. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.

Zhengyan Zhang, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. 2022. MoEfication: Transformer feed-forward layers are mixtures of experts. In *Findings of ACL*, pages 877–890.

Zhengyan Zhang, Fanchao Qi, Zhiyuan Liu, Qun Liu, and Maosong Sun. 2021. Know what you don't need: Single-shot meta-pruning for attention heads. volume 2, pages 36–42.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2024. Judging llm-as-a-judge with mt-bench and chatbot arena. *Proceedings of NeurIPS*, 36.

Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2023. Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*.

A Ouroboros does not cause any performance degradation

Model		Yi 34B/6B	DeepSeek 33b/6.7B	CodeLlama 34B/7B
HumanEval	Greedy	18.3	64.6	36.0
	Speculative	18.9	64.6	36.0
	Lookahead	18.9	64.6	36.0
	Ouroboros	18.9	64.6	36.0
MBPP	Greedy	37.8	70.0	46.6
	Speculative	38.9	71.1	46.6
	Lookahead	38.9	71.1	46.6
	Ouroboros	38.9	71.1	46.6

Table 7: Task performance of HumanEval and MBPP of Greedy, Speculative, Lookahead and Ouroboros using greedy generation. We report Accuracy(%) of HumanEval and MBPP.

Model		Yi 34B/6B	Llama 70B/7B
GSM8K	Greedy	61.0	66.0
	Speculative	61.0	66.0
	Lookahead	61.0	66.0
	Ouroboros	61.0	65.0
CNN/DM	Greedy	35.2	37.1
	Speculative	35.1	37.2
	Lookahead	35.3	37.1
	Ouroboros	35.3	37.1
WMT16	Greedy	21.7	31.1
	Speculative	21.9	31.1
	Lookahead	22.0	31.1
	Ouroboros	21.1	31.1

Table 8: Task performance of GSM8K, CNN/DM and WMT16 of Greedy, Speculative, Lookahead and Ouroboros using greedy generation. We report Accuracy of GSM8K(%), Rouge1 score(%) of CNN/DM and BLEU score(%) of WMT16.

Table 7, 8 and 9 show the task performance of the experiments shown in Figure 5 and Table 2. Theoreotically, Speculative Decoding, Lookahead Decoding and Ouroboros generates the same token sequences compared with vanilla Decoding. We observe no overall performance degradation of Ouroboros from Table 7, 8 and 9. The small difference in task performance among vanilla, Speculative, Lookahead and Ouroboros is caused by floating point error, since the calculations are not fully the same between the methods (especially the calculation order of attention). The experimental results show that the calculation error would not cause notable performance degradation.

Model		Yi 34B/6B
HumanEval	Random Sampling	15.9
	Speculative	18.9
	Lookahead	18.9
	Ouroboros	17.7
MBPP	Random Sampling	36.7
	Speculative	33.3
	Lookahead	35.6
	Ouroboros	36.7
GSM8K	Random Sampling	67.0
	Speculative	70.0
	Lookahead	69.0
	Ouroboros	69.0
CNN/DM	Random Sampling	34.6
	Speculative	34.0
	Lookahead	34.6
	Ouroboros	35.6
WMT16	Random Sampling	20.6
	Speculative	21.2
	Lookahead	22.7
	Ouroboros	25.0

Table 9: Task performance using random sampling. We report accuracy for HumanEval, MBPP and GSM8K, Rouge 1 score for CNN/DM and BLEU score for WMT16.

B Hyperparameters

W is a hyperparameter used in Lookahead Decoding to generate phrases. Larger W indicates more phrases are generated in each forward of our draft model in Ouroboros, but may also be more time-consuming.

To investigate hyperparameters sensitivity, we conduct a grid search on γ, W and β on GSM8K with Yi-34B/6B. We report token/s of each setting.

	$W=14$			$W=15$			$W=16$		
	$\beta=5$	$\beta=6$	$\beta=7$	$\beta=5$	$\beta=6$	$\beta=7$	$\beta=5$	$\beta=6$	$\beta=7$
$\gamma=3$	28.17	28.07	27.30	27.97	28.23	27.38	28.34	27.64	27.21
$\gamma=4$	28.52	28.42	27.91	29.06	28.77	28.23	29.00	28.68	27.64
$\gamma=5$	28.13	29.32	28.20	28.31	28.96	27.99	27.94	29.03	27.90
token/s std: 0.57							speedup std: 0.04		

Table 10: The results of grid search where $\gamma \in [3,5], W \in [14,16], \beta \in [5,7]$. "std" represents the standard deviation.

Table 10 shows that Ouroboros is stable to hyperparameters. This provides an opportunity to provide a general recipe or conduct heuristic search instead of grid search to find a good combination of hyperparameters within a short time.

B.1 Hyperparameters Recipe

Here, we offer a general recipe for hyperparameter tuning in Table 11. We distinguish downstream tasks into two types: (1) **HH**: High homogeneity between draft and target model, e.g. code genera-

tion; (2) **LH**: Low homogeneity between draft and target model, e.g. machine translation.

Tasks	γ	W	β	K
HH	$7 \sim 14$	$15 \sim 20$	$5 \sim 7$	$3 \sim 5$
LH	$2 \sim 6$	$15 \sim 20$	$5 \sim 7$	$3 \sim 5$

Table 11: An empirical recipe for hyperparameters tuning.

B.2 Hyperparameters Tuning

According to the stability of hyperparameters, we conduct a heuristic search to find a good combination. The algorithm of the heuristic search is described in Algorithm 1.

Algorithm 1: Heuristic Search of Hyperparameters

Input: Task type T, Dataset D, Objective function f.
Output: Hyperparameters γ, W, β, K
$K_0 := 3$ `// K is always 3 in our experiments`
$\hat{W} \sim [15, 20]$
$\hat{\beta} \sim [5, 7]$
if $T = HH$ **then**
  $\hat{\gamma} \sim [7, 14]$
else
  $\hat{\gamma} \sim [2, 6]$
end if
$\gamma_0 := \arg\min_\gamma \text{ClockTime}(f(D; \hat{\gamma}, \hat{W}, \hat{\beta}, K_0))$
$W_0 := \arg\min_W \text{ClockTime}(f(D; \gamma_0, \hat{W}, \hat{\beta}, K_0))$
$\beta_0 := \arg\min_\beta \text{ClockTime}(f(D; \gamma_0, W_0, \hat{\beta}, K_0))$
return $\gamma_0, W_0, \beta_0, K_0$

B.3 Hyperparameters Used in Our Experiment

The hyperparameters of our experiments are shown in Table 12. In Table 4, we use $\gamma = 12, W = 20, \beta = 8$ and $K = 3$. In Figure 6, we use $\gamma = 12, W = 20, \beta = 8$. In Table 17, we use $\gamma = 6, W = 18, \beta = 6$ and $K = 3$. In Table 13, we follow the hyperparameters in Table 12.

C Block Efficiency

In order to show the potential of further systematic optimization, we follow Miao et al. (2023) to test the Block Efficiency, with the hypothesis that the drafting time can be fully ignored. Block efficiency is the theoretical upper bound of the acceleration ratio, without considering any resource constraints.

Definition C.1 (Block Efficiency)**.** Define Block Efficiency η as

$$\eta = \frac{\text{\# Generated tokens}}{\text{\# Target model calls}}$$

Task	Model	Speculative γ	Lookahead W	Lookahead β	Ouroboros γ	Ouroboros W	Ouroboros β	Ouroboros K
HumanEval	Yi-34B/6B	12	16	7	12	20	8	3
	DeepSeek-33b/6.7B	3	16	7	11	20	8	3
	CodeLlama-34B/7B	5	17	5	10	20	7	3
MBPP	Yi-34B/6B	6	16	6	6	18	6	3
	DeepSeek-33b/6.7B	5	15	7	7	15	6	3
	CodeLlama-34B/7B	4	16	6	8	16	7	3
GSM8K	Yi-34B/6B	7	15	6	4	15	7	3
	Llama-70B/7B	7	15	6	6	17	7	3
CNN/DM	Yi-34B/6B	5	15	6	10	15	6	3
	Llama-70B/7B	4	16	5	4	13	6	3
WMT16	Yi-34B/6B	2	15	6	5	16	6	3
	Llama-70B/7B	5	18	7	4	13	6	3

Table 12: Hyperparameters of experiments in Figure 5 and Table 2.

We measure Block Efficiency with Yi-34B/6B on all 5 datasets following the settings of Figure 5 and Table 2. Results are in Table 13.

	Speculative	Lookahead	Ouroboros
HumanEval	11.16	3.08	**13.12**
MBPP	6.14	2.71	**7.43**
GSM8K	**5.23**	2.30	4.65
CNN/DM	4.71	1.70	**7.46**
WMT16	2.41	1.37	**4.05**

Table 13: The Block Efficiency η of Yi-34B/6B on HumanEval, MBPP, GSM8K, CNN/DM and WMT16. The method with largest η is highlighted with the **bold** font, and the second largest result is with the underlined font.

From Tabel 13, Ouroboros has relatively high Block Efficiency in most scenarios, showing that it has a larger ideal speedup ratio that may be achieved by future algorithms or systematic implementation.

D Comparing with other draft-accelerated methods

Cascade Speculative Decoding (Chen et al., 2023b) is a typical method that accelerates the draft model in speculative decoding by further using an even smaller model to draft for the draft model. The comparison between Ouroboros and Cascade is described in Table 14. Results show that Ouroboros outperforms Cascade, showing that the way we accelerate the draft is more efficient.

Cascade performs slower than the original speculative decoding. One possible reason is that the tiny models which are utilized to accelerate the Llama-2-7B model, i.e. Llama-160M (Miao et al., 2023) and TinyLlama-1.1B (Zhang et al., 2024b), are not officially trained by Meta. This has led to discrep-

ancies in the model's output, thereby slowing down the drafting process.

Method	Model Series	token/s
Speculative	7B→70B	16.86
Cascade	160M→7B→70B	7.90
Cascade	1.1B→7B→70B	9.22
Ouroboros	7B→70B	**24.03**

Table 14: Comparing with various cascade versions of speculative decoding, tested on GSM8K.

E Ouroboros under various generation lengths

To investigate the speedup ratio under different generation length limits, we conduct the following experiment with generation length limits 64, 128, 256, and 512. This experiment is performed on GSM8K with Yi-34B/6B, with other hyperparameters aligned with Table 12.

		Greedy	Speculative	Lookahead	Ouroboros
$l = 64$	token/s	14.31	20.18	20.20	**30.99**
	speedup	1.00	1.41	1.41	**2.17**
$l = 128$	token/s	14.57	21.90	22.68	**35.32**
	speedup	1.00	1.50	1.56	**2.42**
$l = 256$	token/s	14.41	22.12	24.58	**38.36**
	speedup	1.00	1.54	1.71	**2.66**
$l = 512$	token/s	15.35	21.45	25.14	**40.94**
	speedup	1.00	1.40	1.64	**2.67**

Table 15: Speedup with multiple generation lengths. "l" represents generation length limit.

	Greedy	Speculative	Lookahead	Ouroboros
$l = 64$	44.51	44.51	44.51	44.51
$l = 128$	60.98	60.98	60.98	60.98
$l = 256$	64.02	64.02	63.41	63.41
$l = 512$	64.63	64.63	64.02	64.63

Table 16: Task performance (accuracy) with multiple generation lengths. "l" represents generation length limit.

From Table 15, Ouroboros outperforms autoregressive decoding, Speculative decoding, and Lookahead decoding under various lengths. In Table 16, short generation length limit ($l = 64, 128$) results in answer truncation, which leads to significant lower task performance. Close task performances and speedup ratios are achieved in $l = 256, 512$, showing that all algorithms halt immediately after generating "<EOS>".

Two conclusions can be obtained. First, the length limits used in our experiments are reasonable, as shorter length limits result in ill-generated answers. Second, phrase repetition in the generation is not the reason why Ouroboros achieves such a high speedup. If so, we would be able to observe lower task performance and much higher speedup from $l = 256$ to $l = 512$.

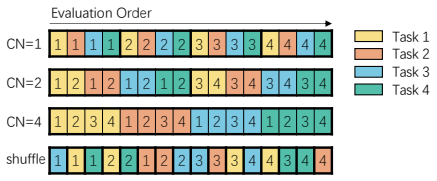

Figure 7: The ablation studies on context locality. In our experiments, the task 1,2,3,4 are MBPP, GSM8K, CNN/DM, and WMT16, respectively.

F Context Locality

We further study how context locality accelerate generation through the reused phases from history.

Context locality remains very high when continually generating within one task, but may vary across different datasets. For example, the phrases pool should contain code pieces when running on code generation datasets such as HumanEval and MBPP, but it might contain natural language pieces when generating text. It remains a question that whether one type of language pieces could accelerate another type of language's generation. We conduct the following experiments to further investigate to what extent and how context locality can accelerate generation through reusing phrases from history.

We select four datasets: MBPP, GSM8K, CNN/DM, and WMT16, corresponding to code generation, arithmetic reasoning, document summarization, and machine translation, and sample 20 entries from each dataset to organize a new dataset containing different domains. We evaluate Ouroboros with multiple evaluation order, from which we could change context locality. We define consecutive number CN as how many entries from the same dataset are tested consecutively, as shown in Figure 7. In the "shuffle" configuration, we randomize the order of data entries. Higher CN indicates a better locality, and the "shuffle" configuration should have the worst locality. We then measure the generation speed in different CN configurations or random shuffling.

Table 17 shows that context locality indeed affects the effectiveness of phrases reusing. With phrases reusing, CN=20 leads to faster decoding

Setting	shuffle	CN=20	shuffle	CN=20	CN=10	CN=4	CN = 1
Phrases Reusing	off	off	on	on	on	on	on
token/s	32.68	32.53	35.39	36.00	35.32	34.83	35.36

Table 17: The results of ablation studies on context locality. “CN” stands for consecutive number.

compared to the shuffle setting. However, the effect caused by context locality is smaller than whether to turn on the phrases reusing. The speed of lower CN is similar to the shuffle setting when applying phrases reusing, but both are approximately 3 token/s faster compared to the cold start setting, proving that the phrases pool is still effective across multiple tasks.